

A Finer Ruler for Graph Networks: Grade Them by the Polynomials They Compute

An ICML 2023 paper from Weizmann, MIT, NVIDIA, and Meta gives a complete basis for equivariant graph polynomials, and uses it to build PPGN++, a new state of the art on molecular benchmarks.

If you want to say how powerful a graph neural network (GNN) is, the field's standard answer is the Weisfeiler-Lehman (WL) hierarchy, a family of graph-isomorphism tests. The higher a model sits in the hierarchy, the more graphs it can tell apart, and the more expressive it is presumed to be. WL has become the default vocabulary for talking about GNN expressive power, cited across a huge swath of the literature.

But WL is an awkward ruler in two ways. First, its definition is combinatorial: it tells you which rung a model is stuck on, but not what operation it is missing or what to add to climb higher. For anyone trying to improve a model, that is almost no guidance at all. Second, its gradations are coarse. Many modern GNNs with very different architectures land on the same WL rung, so the test simply cannot separate them.

This ICML 2023 paper swaps the ruler. Instead of asking which isomorphism test a model passes, it asks which equivariant graph polynomials the model can compute. An equivariant polynomial is a polynomial in the adjacency matrix that respects node relabeling, and it is a natural target: subgraph counts and many structural features are exactly such polynomials. The higher the degree a model can compute, the stronger it is, and the polynomials it cannot yet compute form a concrete to-do list of the operations it lacks.

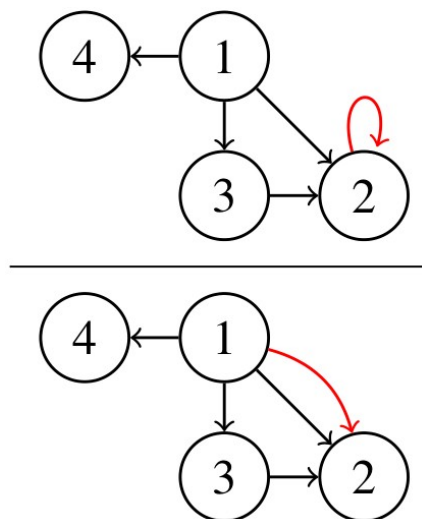


Figure 1. Two example basis elements of equivariant graph polynomials: node-valued (top) and edge-valued (bottom). Each basis element is a small directed multi-graph carrying one special (red) edge, and evaluating it on a graph is a single tensor contraction (an einsum). This is the paper's central intuition: a polynomial is just a small graph.

Paper: <https://arxiv.org/abs/2302.11556>

Code: <https://github.com/omri1348/Equivariant-Polynomialals>

The problem: why WL is not enough

A GNN's expressive power is, at bottom, the set of functions of a graph it can represent, and that set is bounded: no matter how many layers you stack, a standard message-passing network cannot separate certain non-isomorphic graphs. WL characterizes this bound, but as noted it is neither constructive nor fine-grained. What the authors want instead is a ruler that both separates today's models and points directly at the operations a weak model is missing.

Earlier work had only characterized very low-degree equivariant maps, or leaned on WL, leaving a gap: a complete, computation-friendly description of what a GNN should be able to output. Filling that gap is the paper's starting point.

Contribution one: a complete basis, where every element is a multi-graph

The first contribution is a complete basis for all equivariant graph polynomials of degree at most d . Theorem 2.1 states that this basis is enumerated by directed multi-graphs $H = (V, E, (a, b))$, with $|V| \leq \min\{n, 2+2d\}$, $|E| \leq d$, and (a, b) a marked (red) edge. In other words, the abstract space of equivariant polynomials is put into one-to-one correspondence with a collection of small graphs.

And how is each basis element evaluated on a graph? By a single tensor contraction, the einsum expression familiar from deep learning: multiply entries of the adjacency matrix along the black edges of H , then sum over the internal nodes. The simplest example of this is something everyone already knows: ordinary matrix multiplication.

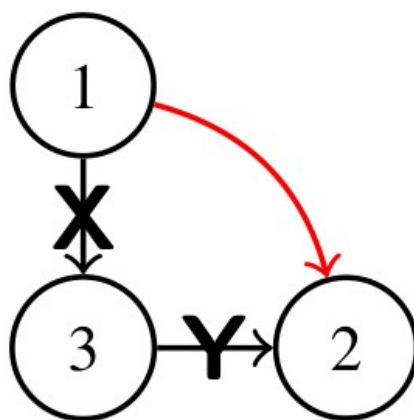


Figure 2. Matrix multiplication XY expressed as a tensor contraction: three nodes, two black edges, and one red output edge, whose einsum is exactly the product XY . The takeaway is that the paper's tensor contractions are nothing exotic; they are just familiar graph linear algebra rewritten in the language of multi-graphs.

How does the construction guarantee equivariance? The authors average an ordinary monomial over all node permutations using the Reynolds operator, and then show that this average equals a sum over the orbit of H , that is, over all the isomorphic copies of H obtained

under permutation. Building an equivariant polynomial thus reduces to enumerating a multi-graph's orbit, a purely combinatorial task.

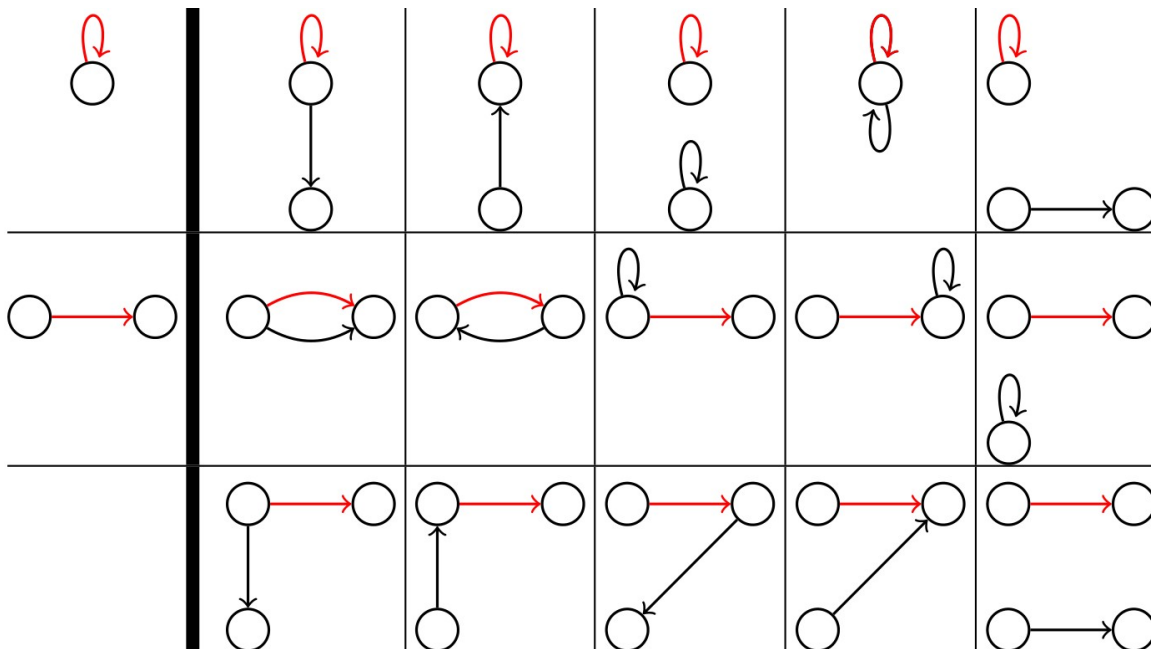


Figure 3. The complete basis of constant (left of the bold line) and linear (right of the bold line) equivariant graph polynomials. Each cell is a small multi-graph, with the red edge marking the output slot. Together the grid is an exhaustive inventory of the low-degree space: every equivariant constant or linear map is a linear combination of these elements.

Contribution two: a linear-time test for a model's polynomial power

With a basis in hand, one can ask a very concrete question: given a GNN, which of these polynomials can it actually compute? The second contribution treats a network as a bank of primitive contractions F , and gives an algorithm (Algorithm 1) that decides, in time linear in $|V|$ and $|E|$, whether a target polynomial can be assembled by repeatedly contracting nodes.

Applying this tool, the authors tabulate the polynomial power of standard GNNs, and two clean results fall out. A bank that only allows node contractions, F_n , turns out to equal 1-WL exactly; a stronger bank that also allows edge contractions, F_e , equals 2-FWL (better known as 3-WL). The figure below draws the primitive building blocks of both banks.

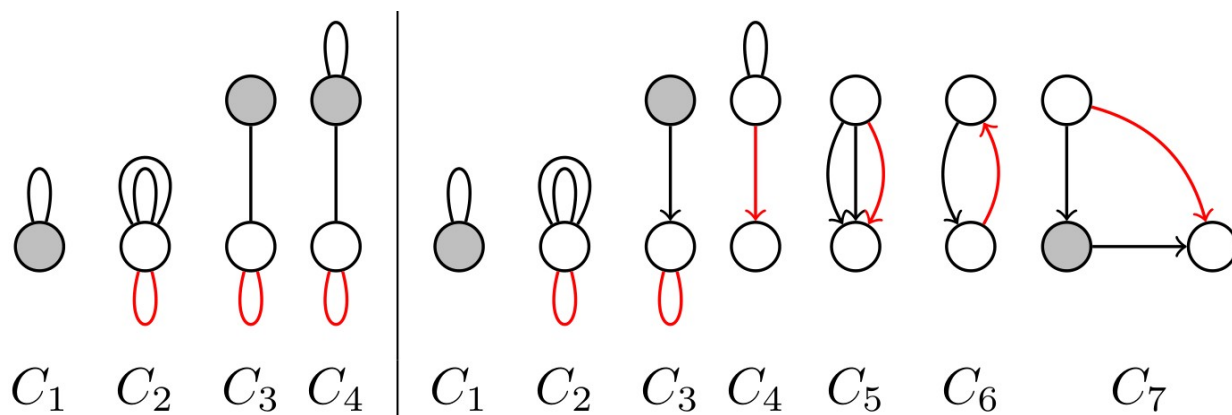


Figure 4 (paper Figure 7). The prototypical node-based (left) and edge-based (right) contraction banks; gray nodes are the ones being contracted. The node-based bank F_n corresponds to 1-WL, and the edge-based bank F_e corresponds to 2-FWL (3-WL). The extra primitives on the edge side are precisely what make it stronger.

This is not a paper exercise. Table 1 quantifies it: on degree-3 polynomials the edge-based model F_e leaves only 0 of 18 basis elements non-computable, whereas the node-based F_n leaves 2 of 8, matching the fact that F_e is 5-node- and 4-edge-polynomial-exact while F_n reaches only 2-node-polynomial-exact.

Contribution three: turning the theory into a model, PPGN++ and precomputed features

Since the theory now says which polynomials are missing, the natural third step is to supply them. On top of the powerful matrix-multiplication network (PPGN/Ring-GNN), the authors do two things. They replace the bloated design that needs separate parameters for each of the 15 linear basis elements with a small set of primitive operations, yielding a leaner, more efficient backbone they call PPGN++. And they precompute high-degree equivariant polynomial features and feed them straight into the network.

That second move is the crux. The polynomial features already encode structure that 3-WL cannot see, and because they are precomputed, the network gains expressive power that is provably strictly beyond 3-WL at almost no runtime cost. A single PPGN++ layer takes the form below, where the three $\bar{m}$ are learnable primitive operations and $\otimes$ denotes elementwise multiplication:

$$Z^{(k+1)} = \bar{m}_3[\bar{m}_1(Z^{(k)}, Z^{(k)T}) \otimes \bar{m}_2(Z^{(k)}), Z^{(k)}]$$

Experiments: state of the art on molecules, past 3-WL on SR graphs

The evaluation spans two settings. For real-world performance, the authors use three standard molecular graph regression datasets: ZINC-12K (about twelve thousand molecules), the larger ZINC-full (over two hundred thousand), and Alchemy, all measured by test mean absolute error (MAE, lower is better), with every model held to a 500K parameter budget following Dwivedi et al. (2020) and Lim et al. (2022). For raw expressive power they use

Strongly Regular (SR) graphs, families notoriously hard to tell apart that 3-WL cannot separate at all, measuring the failure rate at distinguishing non-isomorphic pairs.

Table 1. Test MAE on three molecular graph regression benchmarks (lower is better, all within a 500K parameter budget). PPGN++ with degree-6 polynomial features is best in class on all three.

Model	ZINC-12K	ZINC-full	Alchemy
GIN	0.163	—	0.180
CIN	0.079	0.022	—
PPGN++ (degree 6)	0.071	0.020	0.109

PPGN++ with degree-6 polynomial features takes the top spot on all three: 0.071 on ZINC-12K (versus 0.079 for CIN and 0.163 for GIN), 0.020 on ZINC-full, and 0.109 on Alchemy. Just as striking, the features are model-agnostic. Bolting degree-6 features onto a plain GatedGCN backbone cuts its ZINC-12K MAE from 0.265 to 0.106, so the benefit does not hinge on any one architecture.

Feature degree matters too, and monotonically. On ZINC-12K the base model scores 0.076, degree-5 features bring it to 0.072, and degree-6 to 0.071. On the SR graphs the same trend is far more dramatic.

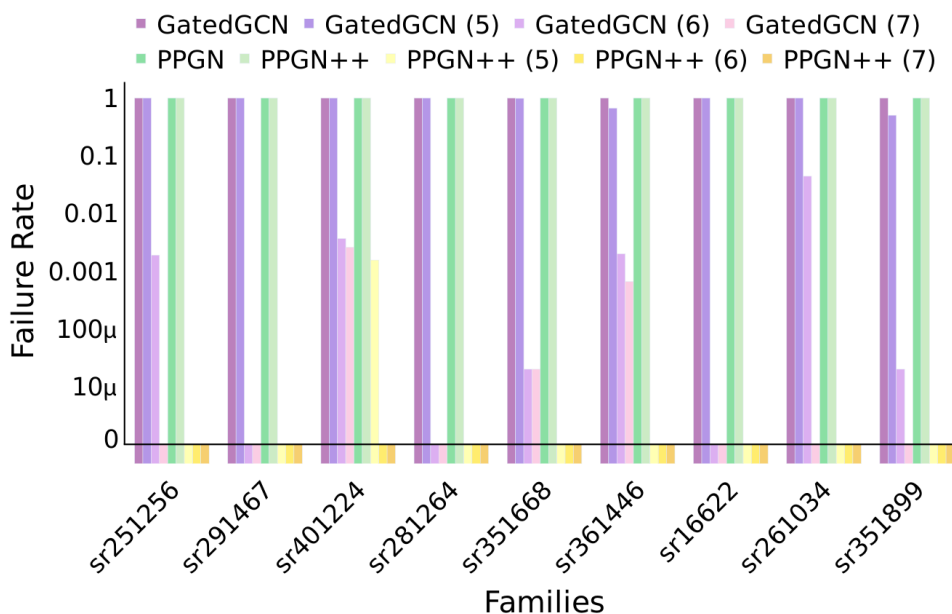


Figure 5 (paper Figure 10). Failure rate at distinguishing non-isomorphic SR graph pairs, by family (log scale, lower is better). For both the GatedGCN and PPGN backbones, adding polynomial features drives the failure rate down by orders of magnitude as the degree climbs from 5 to 7, while 3-WL fails on these graphs entirely. This is direct evidence that precomputed polynomial features push the model past the 3-WL barrier.

Takeaway: making "how expressive?" a checkable, actionable question

The lasting contribution here is turning a vague question, "how expressive is this GNN?", into a concrete and checkable one: "which equivariant graph polynomials can it compute?" Every such polynomial is just a small multi-graph evaluated by a tensor contraction, which gives

both a finer ruler than WL and an explicit, itemized list of exactly which operations a given model still lacks and could be extended to include.

Acting on that list, the authors use precomputed polynomial features to push ordinary graph networks past the 3-WL barrier and to state-of-the-art accuracy on molecular property prediction, all within a modest 500K parameter budget. They are candid about what comes next, including working with a generating set of polynomials rather than the full basis to reduce how many features the network ingests, and incorporating currently-uncomputable polynomials as in-layer computational primitives rather than as precomputed features. For anyone working on graph learning, the paper turns "make the model stronger" from guesswork into a concrete engineering task with an actual checklist to work through.

It is worth being precise about the cost. Because the polynomial features are precomputed once, before training, and passed in as extra inputs, the gain in expressive power does not turn into a heavier per-step runtime, and every model reported in the paper still fits inside the same 500K parameter budget as its baselines. Paired with the linear-time computability algorithm, this is what lets the central question, whether a given network can compute a given polynomial, become something a practitioner can check directly and then act on, instead of a property that can only ever be argued about at the level of intuition and worst-case examples.